



Async

Asynchronous Task Scheduling

Default @Scheduled

By default, the tasks are blocking. This will NOT run every 5 seconds.

```
@Scheduled(fixedRate = 1000)
public void longRunningTask() throws InterruptedException {
    log.info("Starting a task: {}", Thread.currentThread().getName());
    Thread.sleep( millis: 5000); // Simulates a long-running task
    log.info("Ending the task: {}", Thread.currentThread().getName());
}
```

Async Task Scheduling

```
@Component
public class MyScheduledTasks {
    @Scheduled(cron = "*/* * * * *") // Execute every 1 second
    @Async // Execute in a separate thread
    public void task1() {
        // Task logic goes here
        System.out.println("Task 1 executed.");
    }

    @Scheduled(cron = "*/5 * * * *") // Execute every 5 seconds
    @Async // Execute in a separate thread
    public void task2() {
        // Task logic goes here
        System.out.println("Task 2 executed.");
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            // Handle interrupted exception
            e.printStackTrace();
        }
    }
}
```

To avoid blocking the execution of task1() due to the potentially long-running task2(), you can configure them to run in separate threads. You can achieve this by setting the @Async annotation on the methods.

You also need to @EnableAsync to use it.

Custom Executor

When using the `@Async` annotation, Spring by default creates an Executor that has no limit on the number of threads it can create. As a result, if your job is triggered more frequently than it completes, it may lead to memory leaks due to continuously created threads.

However, you can specify a custom Executor bean to be used instead by providing its bean name as an argument to `@Async`.

```
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "jobExecutor")
    public Executor jobExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(20);
        executor.setQueueCapacity(30);
        executor.setThreadNamePrefix("jobExecutor-");
        executor.initialize();
        return executor;
    }
}
```

